

项目技术报告

封面

项目名称：Agentv2

研发单位：个人开发者

编制日期：2026年5月14日

版本号：1.0.0

目录

- 项目概述
- 系统整体架构设计
- 前端层详细设计
- 后端层设计
- 数据层设计
- 工具与插件生态
- 记忆与上下文管理
- 安全与合规设计
- 测试与运维方案

页眉：项目名称 - Agentv2

页脚：页码 | 编制日期：2026年5月14日

项目概述

项目背景

Agentv2是一款基于uni-app框架开发的跨平台AI助手应用，主要面向移动端用户提供智能对话服务。项目集成了AI聊天功能、钱包管理、用户认证等模块，旨在为用户提供便捷的AI交互体验和基本的金融管理工具。该项目源于对AI技术在移动应用中应用的探索，结合了聊天机器人与钱包功能的创新融合。

建设目标

项目的建设目标包括：

- 提供流畅的AI对话界面，支持文本和图片输入。
- 实现用户钱包余额查询和账单记录展示。
- 支持用户注册、登录和密码找回功能。
- 确保跨平台兼容性，包括微信小程序、App等。
- 提供良好的用户体验，包括侧边栏导航、历史会话管理等。

业务价值

Agentv2的应用具有以下业务价值：

- 为用户提供AI驱动的智能助手服务，提升日常生活效率。
- 集成钱包功能，实现AI与金融服务的结合，拓展应用场景。
- 通过跨平台部署，覆盖更广泛的用户群体。
- 提供完整的用户管理体系，支持个性化服务。

整体功能范围

项目的主要功能模块包括：

- 用户认证模块：登录、注册、密码找回。
- AI聊天模块：文本对话、图片上传解析、模型切换。
- 钱包模块：余额查询、账单记录展示。
- 个人中心：用户信息管理。
- 历史会话：会话记录查看和切换。

技术栈整体说明

项目采用uni-app框架开发，支持Vue.js语法，兼容Vue 2和Vue 3。样式使用SCSS预处理器，UI组件基于uni-ui库。网络请求使用uni.request API，后端接口基于RESTful设计。项目支持热更新和跨平台打包。

系统整体架构设计

整体分层架构

系统采用前后端分离架构：

- **前端层**：uni-app框架，负责用户界面和交互逻辑。
- **后端层**：提供API服务，包括AI聊天、用户认证、钱包管理等。
- **数据层**：存储用户数据、会话记录、钱包信息等。

业务架构

业务架构围绕用户为中心，包括：

- 用户管理：认证、资料管理。
- AI服务：对话处理、模型管理。
- 金融服务：钱包余额、账单管理。
- 会话管理：历史记录、上下文保持。

前后端交互架构

前端通过HTTP请求与后端API交互：

- 请求方式：GET、POST等。
- 数据格式：JSON。
- 认证：Bearer Token或X-Username头。
- 错误处理：统一的状态码和错误信息。

模块划分

项目模块划分如下：

- 登录注册模块（login.vue, register.vue）。
- 主聊天模块（index.vue）。
- 钱包模块（wallet.vue）。
- 个人中心模块（profile.vue）。
- 关于页面（about.vue）。

整体流程逻辑

用户流程：

1. 启动应用，进入登录页面。
2. 登录成功后进入主页面。
3. 在主页面进行AI对话或查看钱包。
4. 可通过侧边栏切换到其他功能。

前端层详细设计

技术栈

- 框架：uni-app (基于Vue.js)。
- 语言：JavaScript (ES6+)。
- 样式：SCSS。
- UI库：uni-ui组件库。
- 构建工具：HBuilderX。

项目目录结构

- pages/: 页面文件
 - index/: 主页面
 - login/: 登录页面
 - register/: 注册页面
 - wallet/: 钱包页面
 - profile/: 个人中心
 - about/: 关于页面
- config/: 配置文件
 - api.js: API配置
- static/: 静态资源
 - customicons.css: 自定义图标
- uni_modules/: uni-ui组件
- App.vue: 根组件
- main.js: 入口文件
- pages.json: 页面配置
- manifest.json: 应用配置

页面路由设计

路由配置在pages.json中：

- 登录页：/pages/login/login
- 主页：/pages/index/index
- 注册页：/pages/register/register
- 钱包页：/pages/wallet/wallet
- 个人中心：/pages/profile/profile
- 关于页：/pages/about/about

组件设计

- 使用uni-ui组件库，如uni-card、uni-list等。
- 自定义组件：侧边栏、钱包悬浮球、聊天列表等。
- 组件复用：通过easycom自动导入uni-组件。

全局样式

- 全局样式文件：uni.scss
- 自定义图标：customicons.css
- 主题色： #F8F8F8 背景色
- 响应式设计：使用rpx单位适配不同屏幕。

状态管理

- 使用Vue实例data管理组件状态。
- 全局状态：通过uni.getStorageSync存储用户信息、会话状态等。
- 组件间通信：通过事件总线或props。

页面业务逻辑

- **登录页面**：用户名密码登录，找回密码功能。
- **主页**：AI聊天界面，支持文本和图片输入，模型切换，历史会话。
- **钱包页面**：显示余额和账单列表，支持筛选。
- **注册页面**：用户注册。
- **个人中心**：用户信息展示。
- **关于页面**：应用信息。

交互设计

- 侧边栏滑动菜单。
- 钱包悬浮球拖拽。
- 聊天列表滚动。
- 弹窗对话框。

适配兼容方案

- 使用uni-app的条件编译支持多平台。
- rpx单位自动适配屏幕。
- 兼容Vue 2和Vue 3。

后端层设计

后端接口架构

后端提供RESTful API，包括：

- 认证接口：/auth/login, /auth/register等。
- 聊天接口：/user/chat/send, /user/chat/threads等。
- 钱包接口：/user/wallet, /api/wallet/bills等。

接口规范

- 请求方法：GET, POST。
- 认证：Bearer Token或X-Username。
- 响应格式：{code, msg, data}。
- 错误码：0成功，-1失败等。

请求响应设计

- 请求头：Content-Type: application/json。
- 响应体：统一的JSON结构。
- 分页：page, page_size参数。

业务服务分层

- 控制器层：处理请求路由。
- 服务层：业务逻辑处理。
- 数据访问层：数据库操作。

数据层设计

数据流转流程

- 前端发起请求 -> 后端处理 -> 数据库查询/更新 -> 返回数据 -> 前端渲染。

前端数据结构定义

- 用户信息：username, nickname, avatar, role_code。
- 聊天消息：role, content, image。
- 钱包信息：cash_balance, bank_balance, debt_amount。
- 账单：id, type, amount, description, created_at。

接口数据映射

- API响应数据直接映射到组件data。
- 使用computed属性处理数据格式化。

缓存策略

- 本地存储：uni.setStorageSync存储用户信息。
- 会话缓存：页面间传递数据。

数据持久化方案

- 后端数据库存储用户数据、会话记录。
- 前端不进行本地数据持久化，仅缓存必要信息。

工具与插件生态

项目用到的第三方组件

- uni-ui：UI组件库，包括uni-card, uni-list等。
- uni-scss：样式预处理器。
- customicons.css：自定义图标字体。

UI库

- uni-ui：提供丰富的组件，如按钮、列表、弹窗等。

工具函数

- buildApiUrl：构建API URL。
- formatMoney：格式化金额。
- formatTime：格式化时间。

脚手架

- uni-app CLI：项目初始化和构建。

依赖包

- Vue.js：前端框架。
- uni-app框架：跨平台开发。

自定义工具类

- API配置类：config/api.js。
- 公共方法：组件内methods。

封装公共方法

- 网络请求封装：uni.request调用。
- 数据处理：formatMoney, formatTime等。

记忆与上下文管理

项目全局状态管理

- 使用uni.getStorageSync管理全局状态，如用户信息、侧边栏状态。

页面上下文传递

- 通过路由参数传递数据。
- 使用uni.setStorageSync跨页面共享数据。

路由参数管理

- 页面跳转时传递参数，如uni.navigateTo({url: '/pages/profile/profile'})。

本地存储/会话存储

- uni.setStorageSync：持久化存储。
- 页面data：临时状态。

全局变量与上下文生命周期设计

- 全局变量：API_BASE_URL。
- 上下文生命周期：页面onShow/onHide管理状态。

安全与合规设计

前端请求安全

- 使用HTTPS协议。
- 请求头包含认证信息。

接口请求拦截

- 统一错误处理。
- 请求超时设置。

响应拦截

- 检查响应状态码。
- 处理业务错误。

权限控制

- 登录状态检查。
- 路由守卫：未登录跳转登录页。

路由守卫

- 在页面onShow检查登录状态。

敏感数据处理

- 密码输入类型password。
- 不存储敏感信息在本地。

合规性设计

- 用户协议：注册时同意条款。
- 数据隐私：最小化数据收集。

测试与运维方案

功能测试要点

- 用户登录注册流程。
- AI聊天功能。
- 钱包余额和账单展示。
- 图片上传解析。

兼容性测试

- 多平台兼容：微信小程序、App等。
- 不同屏幕尺寸适配。

打包构建部署流程

- 使用HBuilderX打包。
- 发布到各平台应用市场。

环境配置

- 开发环境：本地API。
- 生产环境：线上API。

版本管理

- 使用Git版本控制。
- 版本号在manifest.json中管理。

日常运维维护方案

- 监控API响应。
- 用户反馈处理。
- 定期更新依赖包。